Комплект материалов курса MLOps 2026 2027

Программа курса методические указания к лабораторным работам и правила командной работы

Для потока не менее 30 студентов
Команды по 2–5 человек
13 лабораторных работ и итоговый проект
Версия от 24 сентября 2026 года

Назначение комплекта

Тринадцать лабораторных работ образуют последовательный практический контур курса, а отдельный итоговый командный проект объединяет управление данными, обучение и регистрацию модели, создание сервиса, автоматизацию проверок, развертывание и наблюдаемость. Командная оценка дополняется обязательным подтверждением индивидуального вклада, поэтому общий результат не заменяет личное освоение материала.

Основная конфигурация потока 30 студентов распределяются в 8 команд: шесть команд по 4 человека и две команды по 3 человека. При другом составе преподаватель использует таблицу вместимости и сохраняет размер команды от 2 до 5 человек.

Принцип обновления Инструмент не становится целью занятия. Каждая тема связана с проверяемым инженерным результатом, решением о выпуске или эксплуатационным риском.

Краткий аудит материалов 2025 года

Наблюдение	Риск для нового потока	Решение 2026–2027
В СКИФ размещены 13 лабораторных и отдельная итоговая защита	Структура 14 лабораторных расходится с фактическими элементами курса	13 ЛР плюс итоговый командный проект; Responsible AI и LLMOps включены в release review
В CI используются actions checkout v3 и setup python v4	Материалы быстро устаревают и не совпадают с официальными примерами	Версии сверяются перед семестром; стартовый шаблон использует актуальные major-версии Actions
Model Registry описан через staging и production stages	Stages объявлены deprecated в MLflow	Aliases champion и candidate, tags validation_status и проверяемое решение о выпуске
Airflow показан через старые импорты и BashOperator	Студенты закрепляют устаревший интерфейс	Airflow 3, публичный airflow.sdk и идемпотентные TaskFlow-задачи
FastAPI принимает payload dict без строгой схемы	Ошибки входа обнаруживаются поздно, контракт API не фиксирован	Pydantic-модели, OpenAPI, health, readiness, request_id и отрицательные тесты
Контейнер и Kubernetes оцениваются главным образом по факту запуска	Не формируются навыки безопасного выпуска	Non-root, probes, resource limits, Pod Security, SBOM и сканирование
Мониторинг ограничен Prometheus и Grafana	Нет связи между запросом, логом и поведением модели	Метрики, структурированные логи, traces, SLI/SLO и runbook
Лабораторные сдаются в общую ветку без обязательного peer review	Для потока 30+ человек невозможно надежно подтвердить вклад	Отдельный репозиторий команды, pull request, review, ротация ролей и evidence card

Результаты обучения

После курса студент должен уметь:

- воспроизвести эксперимент по зафиксированным версиям кода, данных, зависимостей и конфигурации
- построить pipeline подготовки данных, обучения, оценки и регистрации модели с наблюдаемыми артефактами
- создать тестируемый API, безопасный контейнер и базовый Kubernetes deployment
- настроить проверки pull request, версионированную сборку и управляемое продвижение модели
- интерпретировать метрики сервиса, модели и данных, отличать дрейф от деградации качества
- оформить runbook, выполнить rollback и провести краткий разбор учебного инцидента
- документировать назначение модели, ограничения, риски данных и условия человеческого контроля
- работать в команде через небольшие pull request, code review, ротацию ролей и прозрачное подтверждение вклада
- объяснить различия MLOps и LLMOps, включая трассировку, eval datasets, стоимость и риски генеративных систем

Структура семестра

Неделя	Лекция	Основное содержание	Лабораторная	Контрольный результат
1	Введение в MLOps	Жизненный цикл ML-системы, роли и командный workflow	ЛР 1. Репозиторий команды и среда	Команда и репозиторий готовы
2	Управление данными и репозитории	Источники, схемы, data contract и качество	ЛР 2. Контракт данных	Data card и проверки качества
3	Управление экспериментами	DVC, предобработка и воспроизводимый pipeline	ЛР 3. DVC pipeline	dvc repro восстанавливает результат
4	Контейнеризация ML-приложений	MLflow Tracking и подготовка модели к поставке	ЛР 4. Tracking экспериментов	Сравнимые runs и кандидат
5	Оркестрация и CI/CD для ML	Оценка модели и управление версиями	ЛР 5. Оценка и Registry	Модель зарегистрирована через aliases/tags
6	Автоматизация пайплайнов ML	FastAPI, Docker и контракт сервиса	ЛР 6. REST API и контейнер	Проверяемый непривилегированный образ
7	Развертывание ML-моделей	Тестирование и автоматические проверки PR	ЛР 7. Тесты и GitHub Actions	Обязательный зеленый CI

Неделя	Лекция	Основное содержание	Лабораторная	Контрольный результат
8	Микросервисы и масштабирование	Airflow 3 и управляемый pipeline	ЛР 8. Airflow DAG	Идемпотентный run с артефактами
9	Мониторинг ML-систем	Feature Store и временная корректность	ЛР 9. Feast	Point-in-time корректные признаки
10	Тестирование ML-моделей	Kubernetes, probes, ресурсы и безопасность	ЛР 10. Kubernetes	Deployment с проверяемым rollback
11	Управление версиями моделей	Метрики, логи, traces и SLI/SLO	ЛР 11. Monitoring	Dashboard и alert с runbook
12	Этические и юридические аспекты	Drift, retraining gate и incident response	ЛР 12. Drift	Управляемое решение о реакции
13	Feature Store и продвинутые инструменты	Полный release pipeline и rollback	ЛР 13. CI/CD	Неизменяемый release в кластере
14	Итоги курса и практические кейсы	Responsible AI, LLMOps по сценарию и интеграция	Итоговый командный проект	End-to-end демонстрация и защита

Командная модель для потока от 30 студентов

Размер и количество команд

Студентов	Рекомендуемое распределение	Команд	Комментарий
30	6 команд по 4 и 2 команды по 3	8	Базовая конфигурация
31	7 команд по 4 и 1 команда из 3	8	Равномерная нагрузка
32	8 команд по 4	8	Предпочтительная конфигурация
33	7 команд по 4 и 1 команда из 5	8	Одна команда максимального размера
34	6 команд по 4 и 2 команды по 5	8	Все команды в диапазоне
35	5 команд по 4 и 3 команды по 5	8	Все команды в диапазоне
36	4 команды по 4 и 4 команды по 5	8	Все команды в диапазоне

Студентов	Рекомендуемое распределение	Команд	Комментарий
37	3 команды по 4 и 5 команд по 5	8	Все команды в диапазоне
38	2 команды по 4 и 6 команд по 5	8	Все команды в диапазоне
39	1 команда из 4 и 7 команд по 5	8	Все команды в диапазоне
40	8 команд по 5	8	Верхняя граница для восьми проектных треков

Для состава свыше 40 студентов создается девятая команда. Команда из 2 человек допускается при позднем отчислении или переводе, но преподаватель сокращает необязательный scope, не снижая обязательные критерии качества.

Формирование команд

1. До первого занятия собрать короткую анкету: Python, Git, Docker, Linux, ML, предпочтительный проектный трек и доступность оборудования.
2. Сформировать команды преподавателем, распределив опытных студентов и не допуская концентрации всех навыков в одной группе.
3. На первом занятии команда подписывает рабочее соглашение: канал связи, время ответа, правила review, Definition of Done и порядок разрешения конфликтов.
4. После второй лабораторной состав фиксируется. Замена участника оформляется записью в журнале и обновлением CODEOWNERS и таблицы ролей.

Роли и ротация

Роль	Ответственность	Подтверждение вклада
Release lead	План итерации, интеграция и release notes	Связанные issue, merge и tag
Data and model engineer	Данные, признаки, обучение и оценка	DVC или MLflow artifacts и review
Platform engineer	Контейнер, CI, deployment и секреты	Workflow run, image digest и manifests
Quality and observability engineer	Тесты, telemetry, drift и runbook	Tests, dashboard, alert или incident record
Risk reviewer	Model card, risk register и release decision	Review comment и evidence links

В команде из 2–4 человек роли совмещаются. Роль release lead меняется не реже одного раза в две лабораторные. Один студент не может оставаться единственным владельцем кода модели или инфраструктуры более двух последовательных итераций.

Рабочий процесс команды

- Каждая задача имеет issue, владельца, критерий приемки и ссылку на лабораторную работу.
- Изменение поступает в основную ветку только через pull request и минимум одно содержательное review другого участника.
- Pull request должен быть небольшим. Изменения данных, модели и deployment не объединяются без необходимости.
- Секреты не хранятся в Git. Учебные доступы заменяются переменными окружения и примером .env.example.
- К концу лабораторной каждый студент добавляет evidence card: вклад, ссылка, принятое решение, ошибка или риск и чему научился.

Оценивание

Компонент	Вес	Как подтверждается
Командные результаты лабораторных работ	40 процентов	Артефакты, приемочные проверки и демонстрация
Индивидуальный вклад в лабораторные	20 процентов	Commits, reviews, evidence cards и ответы на защите
Два контрольных рубежа	10 процентов	Архитектурный review после ЛР 5 и release readiness после ЛР 10
Самоконтроль и короткие тесты	10 процентов	Индивидуальные ответы по модулям
Итоговый проект и защита	20 процентов	Работающая система, сценарий отказа, документация и индивидуальный вопрос

Условие допуска Минимум 60 процентов по командным результатам и минимум 50 процентов по индивидуальным компонентам. Студент с нулевым подтвержденным вкладом за две последовательные лабораторные проходит индивидуальное восстановительное задание.

Единая рубрика лабораторной работы

Критерий	Баллы	Пояснение
Работоспособность	0–3	Основной сценарий выполняется, ошибки обрабатываются
Воспроизводимость	0–2	Версии входов и зависимостей зафиксированы, запуск документирован
Качество и безопасность	0–2	Есть содержательные проверки и соблюдены требования конкретной работы
Командный процесс	0–1	Issue, pull request и review прослеживаются

Критерий	Баллы	Пояснение
Индивидуальная защита	0–2	Студент объясняет вклад и инженерное решение

Лабораторная засчитывается при результате не ниже 6 из 10 и отсутствии стоп-фактора: опубликованный секрет, невоспроизводимый результат, отсутствующий обязательный артефакт или подмена индивидуального вклада чужим commit.

Definition of Done для каждой лабораторной

- issue содержит критерии приемки и ссылку на номер лабораторной работы
- изменение прошло pull request и содержательное review другого участника
- обязательные автоматические проверки завершились успешно
- README или профильная документация описывают новый способ запуска
- артефакты имеют версию или однозначную связь с commit SHA
- в журнале команды записаны результат демонстрации и известные ограничения

Форма индивидуальной evidence card

Поле	Содержание
Вклад	Конкретное изменение или review, без общего описания работы команды
Ссылка	Commit, pull request, issue, run, dashboard или документ
Решение	Какой вариант выбран и по какой проверяемой причине
Риск или ошибка	Что могло пойти неверно и как это обнаруживается
Результат обучения	Что студент теперь может объяснить или повторить самостоятельно

Методические указания к лабораторным работам

Все лабораторные продолжают один проект. Команда может выбрать классическую ML задачу или согласованный LLM трек. Для сравнимости оценки обязательный контракт артефактов одинаков для всех треков.

Лабораторная работа 1 Репозиторий команды и воспроизводимая среда

Цель: организовать командную работу в GitHub и зафиксировать воспроизводимый запуск учебного ML-проекта.

Основные задачи

- Создать команду из 2–5 человек, назначить стартовые роли и создать репозиторий из шаблона курса.
- Настроить protected main, pull request, review другого участника, Issues и шаблоны сдачи.
- Зафиксировать зависимости, команды установки, тестирования и запуска в README.
- Оформить ADR 0001 с границами проекта, данными, метрикой и основными рисками.

Артефакты: README, CODEOWNERS, CONTRIBUTING, ADR, board задач и принятый pull request.

Приемка: новый участник воспроизводит запуск по README; прямой push в main запрещен; CI выполняет базовые проверки.

Индивидуальное подтверждение: Issue, commit или review и краткое объяснение выбранного командного процесса.

Лабораторная работа 2 Управление данными и репозиториями

Цель: описать данные как управляемый актив и определить проверяемый контракт качества.

Основные задачи

- Зафиксировать источник, лицензию, схему, ключи, временные поля и единицы измерения.
- Определить правила пропусков, дубликатов, диапазонов и разделения train/test без утечки.
- Создать data card и набор автоматических проверок входного набора.
- Исключить данные, модели и секреты из Git; подготовить разрешенный учебный sample.

Артефакты: data card, schema или contract, validation report, разрешенный sample и обновленный README.

Приемка: проверки обнаруживают минимум два намеренно внесенных нарушения, а происхождение набора прослеживается.

Индивидуальное подтверждение: описание одной проверки данных и риска, который она предотвращает.

Лабораторная работа 3 Предобработка и DVC pipeline

Цель: реализовать воспроизводимую предобработку и версионирование данных средствами DVC.

Основные задачи

- Инициализировать DVC, настроить учебное remote-хранилище и добавить исходные данные.
- Описать стадии preprocess, validate и split в dvc.yaml с явными deps, outs и params.
- Выполнить dvc repro и зафиксировать dvc.lock, параметры и validation report.
- Проверить восстановление pipeline после удаления производных артефактов и повторного dvc pull.

Артефакты: dvc.yaml, dvc.lock, params.yaml, DVC metadata и отчет валидации.

Приемка: pipeline полностью воспроизводится, изменение параметра перезапускает только зависимые стадии.

Индивидуальное подтверждение: студент объясняет одну связь deps/outs и показывает соответствующий DVC run.

Лабораторная работа 4 Обучение и tracking экспериментов

Цель: получить сравнимые ML-эксперименты и связать каждый run с версиями кода и данных.

Основные задачи

- Реализовать baseline и минимум один обоснованный вариант модели.
- Логировать в MLflow параметры, метрики, dataset version, commit SHA и артефакты.
- Добавить signature, input example и фиксированный способ разбиения данных.
- Сравнить runs по заранее выбранной метрике и оформить решение в ADR.

Артефакты: MLflow experiment, таблица сравнения runs, модели и ADR выбора кандидата.

Приемка: повторный run с теми же входами дает сопоставимый результат, а выбранная модель прослеживается до данных.

Индивидуальное подтверждение: студент защищает один эксперимент и причину изменения параметра или признака.

Лабораторная работа 5 Оценка модели и Model Registry

Цель: подготовить содержательный отчет качества и зарегистрировать проверенную версию модели.

Основные задачи

- Рассчитать ROC AUC, precision, recall, confusion matrix и метрику, связанную со сценарием проекта.
- Сохранить eval report и отрицательные примеры как артефакты MLflow run.
- Зарегистрировать модель с signature и tags validation_status, owner и dataset_version.
- Назначить alias candidate; deprecated stages Staging и Production не использовать.

Артефакты: reports/eval, зарегистрированная модель, tags, alias candidate и release decision.

Приемка: метрики воспроизводятся, ограничения модели описаны, Registry связан с конкретным run.

Индивидуальное подтверждение: студент объясняет компромисс precision/recall и правило продвижения кандидата.

Лабораторная работа 6 REST API и безопасный контейнер

Цель: реализовать типизированный inference API и упаковать его в воспроизводимый контейнер.

Основные задачи

- Создать FastAPI endpoints /predict, /healthz, /readyz и /metrics с Pydantic-контрактами.
- Возвращать версию модели и request_id; корректно обрабатывать невалидный вход.
- Собрать Docker image с фиксируемой базой, непривилегированным пользователем и .dockerignore.
- Выполнить smoke tests, сформировать SBOM и проверить образ сканером уязвимостей.

Артефакты: API, OpenAPI schema, Dockerfile, image digest, SBOM и smoke-test log.

Приемка: контейнер запускается без root, readiness зависит от модели, отрицательные запросы дают ожидаемые коды.

Индивидуальное подтверждение: студент показывает свой endpoint, тест или hardening-настройку.

Лабораторная работа 7 Тестирование и базовый CI

Цель: настроить GitHub Actions, который проверяет код, данные и контракт сервиса при каждом pull request.

Основные задачи

Добавить unit, contract, integration и model smoke tests; формальный assert True запрещен.

Добавить Ruff и pytest в одинаковую локальную и CI-конфигурацию.

Настроить workflow pull_request и push с актуальными major-версиями официальных Actions.

Сделать проверки обязательными для merge и сохранить ссылку на успешный run.

Артефакты: tests, pyproject.toml, .github/workflows/ci.yml и успешный CI run.

Приемка: намеренная поломка теста блокирует merge; секреты не печатаются и не хранятся в репозитории.

Индивидуальное подтверждение: студент объясняет риск, который контролирует разработанный им тест.

Лабораторная работа 8 Оркестрация pipeline в Airflow 3

Цель: связать подготовку данных, обучение, оценку и регистрацию модели в управляемый DAG.

Основные задачи

Создать DAG через публичный интерфейс airflow.sdk; внутренние импорты Airflow не использовать.

Выделить задачи preprocess, validate, train, evaluate и register с явными зависимостями.

Настроить retries, timeouts, параметры и идемпотентный повторный запуск.

Передавать через XCom только небольшие метаданные, а артефакты хранить во внешнем хранилище.

Артефакты: Airflow 3 DAG, успешный run, task logs и ссылки на созданные артефакты.

Приемка: повторный запуск безопасен, сбой одной задачи диагностируется по логам и не регистрирует непроверенную модель.

Индивидуальное подтверждение: студент объясняет одну задачу, retry policy и контракт ее входов/выходов.

Лабораторная работа 9 Feature Store и point in time корректность

Цель: вынести признаки в Feature Store и исключить временную утечку при формировании обучающей выборки.

Основные задачи

Создать Feast repository, entity, feature view и offline source с event timestamp.

Получить historical features для обучения с point-in-time join.

Добавить тест, который обнаруживает использование будущих значений признака.

По возможности материализовать признаки и продемонстрировать online retrieval для API.

Артефакты: feature_repo, feature definitions, historical retrieval и leakage test.

Приемка: обучающая выборка собирается через Feature Store, а время события и происхождение признаков объяснимы.

Индивидуальное подтверждение: студент защищает одну feature definition и ее временную семантику.

Лабораторная работа 10 Развертывание в Kubernetes

Цель: развернуть API в Minikube или kind и обеспечить проверяемую работоспособность и безопасность workload.

Основные задачи

Создать Deployment и Service, использовать неизменяемый tag или image digest.

Настроить readiness/liveness probes и requests/limits; для HPA определить измеримый сценарий нагрузки.

Добавить runAsNonRoot, запрет privilege escalation, seccomp и сокращенный набор capabilities.

Проверить rollout и выполнить документированный rollback на предыдущий образ.

Артефакты: Kubernetes manifests, rollout status, нагрузочный сценарий и подтвержденный rollback.

Приемка: сервис доступен, неготовый pod не принимает трафик, workload соответствует базовым требованиям Pod Security.

Индивидуальное подтверждение: студент объясняет одну настройку probes, resources или securityContext.

Лабораторная работа 11 Мониторинг сервиса и модели

Цель: сделать состояние ML-сервиса наблюдаемым и связать технические сигналы с поведением модели.

Основные задачи

Экспортировать request count, error rate, latency histogram и ограниченную метрику распределения предсказаний.

Настроить Prometheus target и Grafana dashboard с техническими и ML-показателями.

Добавить структурированные логи с request_id и, по возможности, трассировку OpenTelemetry.

Определить SLI, SLO, alert и владельца реакции; подготовить короткий runbook.

Артефакты: /metrics, Prometheus config, Grafana dashboard JSON, sample logs, SLI/SLO и alert rule.

Приемка: тестовый сбой виден на dashboard и диагностируется без раскрытия чувствительных данных.

Индивидуальное подтверждение: студент проводит один сигнал от запроса до графика или alert.

Лабораторная работа 12 Детекция drift и управляемая реакция

Цель: обнаружить изменение данных или качества и принять контролируемое решение о реакции.

Основные задачи

Сравнить reference и current выборки с помощью Evidently, PSI, KS или обоснованного эквивалента.

Зафиксировать пороги, минимальный объем данных и ограничения статистического вывода.

Включить drift check в Airflow или расписание, но отделить retraining от автоматического release.

Смоделировать инцидент и оформить decision record, runbook и краткий postmortem.

Артефакты: drift report, policy, retraining trigger, quality gate, runbook и postmortem.

Приемка: система корректно различает observe, investigate, retrain и rollback; непроверенная модель не выпускается.

Индивидуальное подтверждение: студент выполняет роль incident commander, investigator, scribe или reviewer.

Лабораторная работа 13 Полный CI CD и release

Цель: собрать проверяемый путь от pull request до обновления учебного Kubernetes-кластера.

Основные задачи

Расширить CI: lint, tests, оценка модели, сборка и сканирование контейнера.

Публиковать образ в GHCR или согласованный registry по неизменяемому tag/digest; secrets хранить только в защищенном хранилище.

После merge обновлять manifest или GitOps-репозиторий и проверять rollout.

Связать release с commit SHA, dataset version, model alias и rollback-процедурой.

Артефакты: CI/CD workflows, image digest, manifests, deployment evidence и release notes.

Приемка: merge запускает контролируемое развертывание, failed gate блокирует release, rollback воспроизводим.

Индивидуальное подтверждение: студент показывает принадлежащий ему этап pipeline и обработку отказа.

Итоговый командный проект и защита

Цель: интегрировать результаты 13 лабораторных в end-to-end ML-систему и подтвердить индивидуальный вклад каждого участника.

Основные задачи

Собрать воспроизводимый путь DVC, Airflow, MLflow Registry, Feature Store, API, Kubernetes, monitoring, drift и CI/CD.

Подготовить README, архитектурную схему, release notes, demo 3–5 минут и отчет о решениях и ограничениях.

Добавить model card, data card и risk register; для LLM-проекта также eval dataset, трассы, стоимость и проверки prompt injection.

Продемонстрировать happy path, один управляемый отказ и rollback; провести устную защиту 5–10 минут.

Артефакты: release tag, image digest, model alias, dataset version, dashboard, runbook, отчет, demo и evidence cards.

Приемка: другой участник воспроизводит систему по README; команда объясняет ограничения, риски и эксплуатационную реакцию.

Индивидуальное подтверждение: случайный вопрос по данным, модели, платформе, наблюдаемости или рискам и ссылка на личный артефакт.

Итоговый проект

Проект должен демонстрировать не сложность модели, а управляемость жизненного цикла. Baseline с сильной воспроизводимостью, тестированием и эксплуатационными артефактами оценивается выше сложной модели без доказуемого процесса.

Обязательный контракт проекта

- README с быстрым стартом и архитектурной схемой
- версии кода, данных, зависимостей и конфигурации
- воспроизводимый train и evaluate pipeline
- experiment tracking и зарегистрированная модель с alias
- набор содержательных тестов и обязательный CI gate
- контейнер без root и отчет проверки зависимостей или образа
- API или batch interface с валидируемым контрактом
- deployment либо полностью воспроизводимый локальный compose профиль
- метрики, структурированные логи и минимум один сценарий диагностики
- drift report, policy реакции и возможность rollback
- model card, data card, risk register и evidence cards участников

Проектные треки для восьми команд

Трек	Задача	Особый эксплуатационный риск
A	Прогноз задержки рейса	Временная утечка и сезонный drift
B	Прогноз спроса	Нестабильность MAPE и праздничные периоды

Трек	Задача	Особый эксплуатационный риск
C	Отток клиентов	Delayed labels и стоимость ошибок
D	Обнаружение аномалий	Редкие события и ложные тревоги
E	Классификация обращений	Сдвиг языка и защита текста
F	Рекомендации учебных материалов	Feedback loop и персональные данные
G	Контроль качества изделий	Class imbalance и изменение камеры
H	LLM помощник по базе знаний	Hallucination, prompt injection, PII и стоимость

Формат защиты

1. Две минуты: задача, пользователи, ограничения и версия релиза.
2. Четыре минуты: путь от данных до работающего сервиса с указанием версий артефактов.
3. Три минуты: наблюдаемость, смоделированный сбой и действие по runbook.
4. Две минуты: ограничения модели, риски и решение о выпуске.
5. До восьми минут: случайные индивидуальные вопросы всем членам команды.

Организация среды и масштабирование поддержки

Уровень	Обязательная среда	Допустимая альтернатива при ограниченных ресурсах
Базовый	Python 3.12, Git, DVC, MLflow, FastAPI, Docker Compose	Podman или локальный Python профиль для ранних работ
Оркестрация	Airflow 3 в Docker Compose	DVC pipeline плюс документированное сравнение с Airflow
Кластер	kind или Minikube на команду	Один общий учебный кластер с namespaces и quotas
Мониторинг	Prometheus и Grafana	Prometheus metrics плюс сохраненный dashboard JSON
Feature Store	Feast local provider	Схема feature definitions и point in time test
LLMOps	Локальная или согласованная модель, трассы и eval dataset	Mock provider без отправки учебных данных во внешний сервис

Преподаватель не должен проверять восемь уникальных стеков. Все команды используют общий базовый шаблон, одинаковые имена команд запуска и единую структуру evidence. Различия допускаются внутри проектного трека, но не в обязательном контракте.

Контрольные рубежи преподавателя

Срок	Что проверяется	Решение
До семестра	Версии образов и actions, доступность шаблонного репозитория, лицензии датасетов	Заморозить tested baseline и опубликовать changelog
После ЛР 2	Состав команд, доступы, data cards, уникальность треков	Зафиксировать команды и scope
После ЛР 5	Архитектура, тесты, контейнер, распределение вклада	Продолжить, сократить scope или назначить корректирующую работу
После ЛР 10	Release readiness, telemetry, риски и технический долг	Допуск к финальному релизу
Перед защитой	Tag, image digest, model alias, dataset version и сценарий rollback	Назначить слот защиты

Карта обновления существующих материалов

Материал 2025	Действие	Приоритет
Программа курса MLOps	Заменить планом из этого комплекта	Обязательно
МУ к лабораторным работам	Заменить заданиями 1–13, итоговым проектом и общей рубрикой	Обязательно
Лекции 1–4	Обновить командный workflow, contracts, MLflow aliases и тестирование	Высокий
Лекции 5–8	Разделить Airflow 3, CI CD CT, serving и стратегии выпуска	Высокий
Лекции 9–11	Добавить logs traces SLO, drift policy и incident response	Высокий
Лекция 12	Обновить правовые сроки на дату занятия и связать governance с release artifacts	Высокий
Лекция 13	Сохранить Feature Store, усилить point in time correctness и lineage	Средний
Финальное занятие	Синхронизировать итоговый проект со СКИФ; включить Responsible AI, LLMOps по сценарию и индивидуальную защиту	Высокий

Материал 2025	Действие	Приоритет
Тесты	Удалить вопросы на запоминание брендов, добавить сценарные вопросы и анализ инцидентов	Средний
Учебный проект mlops flight delay	Заменить dummy test, типизировать API, добавить CI, security и telemetry	Обязательно

Правила ежегодной актуализации

1. За две недели до семестра проверить официальные release notes MLflow, Airflow, GitHub Actions, Kubernetes, DVC, Feast и выбранного monitoring stack.
2. Обновить только tested baseline. Не требовать от студентов newest release без проверки совместимости шаблона.
3. Зафиксировать версии в course.env или lock file и сохранить changelog с причиной изменения.
4. Повторно проверить правовой модуль в день подготовки лекции. Нормативные сроки и требования не переносятся из прошлогодних слайдов автоматически.
5. Прогнать эталонный проект с нуля на чистой машине или runner и сохранить время, требования к памяти и известные ограничения.

Источники для версии 2026 2027

Источник	Адрес	Использование в курсе
MLflow Model Registry Workflows	https://mlflow.org/docs/latest/ml/model-registry/workflow	Deprecated stages, aliases, tags и отдельные окружения
Apache Airflow 3 release notes	https://airflow.apache.org/docs/apache-airflow/3.0.0/release_notes.html	Публичный airflow.sdk и изменения Airflow 3
GitHub Docs Building and testing Python	https://docs.github.com/en/actions/tutorials/build-and-test-code/python	Актуальный пример checkout и setup-python
Kubernetes Pod Security Standards	https://kubernetes.io/docs/concepts/security/pod-security-standards/	Baseline и Restricted profiles
Kubernetes Security Checklist	https://kubernetes.io/docs/concepts/security/security-checklist/	Ресурсы, Pod Security и другие базовые проверки
OpenTelemetry Observability Primer	https://opentelemetry.io/docs/concepts/observability-primer/	Метрики, логи и traces как связанные сигналы
European Commission AI Act	https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai	Сроки применения и риск-ориентированная модель на дату обновления

Источник	Адрес	Использование в курсе
NIST AI RMF Generative AI Profile	https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf	Риски генеративного ИИ
MLflow GenAI overview	https://mlflow.org/docs/latest/genai/overview/	Tracing, eval datasets и контроль качества LLM приложений
Официальное опубликование приказа Роскомнадзора от 19 июня 2025 года номер 140	https://publication.pravo.gov.ru/document/0001202508010002	Требования и методы обезличивания персональных данных
Курс СКИФ Управление процессами машинного обучения MLOps	https://do.skif.donstu.ru/course/view.php?id=7508	Структура 13 лабораторных, итогового проекта, контрольной работы и теста

Правовой раздел предназначен для учебного разбора инженерных обязанностей и не заменяет консультацию юриста или локальные нормативные акты университета.

Чек лист готовности курса к запуску

- Опубликован шаблонный репозиторий с tag semester-2026-2027
- Все команды запуска documented и проверены на чистом окружении
- Подготовлены 8 проектных треков и резервный набор данных
- В журнале есть места минимум для 40 студентов и 10 команд
- Опубликованы рубрики, условия допуска и форма evidence card
- Назначены два контрольных рубежа и слоты финальной защиты
- Проверены требования к оборудованию и общий план для студентов без локального Kubernetes
- Правовой модуль проверен по источникам на дату занятия